



Vupr-QLI: Complex Coherence Projection for Network Intrusion Detection

Reproduces Figures 1-3 for Pattern Recognition submission

Dataset: CIC-IDS2017 - DoS Hulk vs BENIGN

Expected Results: AUC = 0.947, TPR = 0.912, FPR = 0.071

"""

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.metrics import roc_curve, auc
```

```
import os
```

```
import argparse
```

```
# Use clean style for publication
```

```
plt.rcParams['font.family'] = 'Arial'
```

```
plt.rcParams['axes.linewidth'] = 1.2
```

```
def run_vupr_qli(data_dir, attack_label="DoS Hulk", n_samples=10000, seed=0):
```

```
    # =====
```

```
    # 1) LOAD DATA
```

```
    # =====
```

```
    df_mon = pd.read_csv(os.path.join(data_dir, "Monday-WorkingHours.pcap_ISCX.csv"))
```

```
    df_wed = pd.read_csv(os.path.join(data_dir, "Wednesday-workingHours.pcap_ISCX.csv"))
```

```
    df = pd.concat([df_mon, df_wed], ignore_index=True)
```

```
    df.columns = df.columns.str.strip()
```

```
print(f"Loaded: {df.shape[0]} flows")
```

```
# =====
```

2) CLEAN DATA

```
# =====
```

```
df = df.replace([np.inf, -np.inf], np.nan)
```

```
df = df.dropna()
```

```
# =====
```

3) DEFINE FEATURES

```
# =====
```

```
coherence_features = [
```

```
    "Flow Duration",
```

```
    "Total Length of Fwd Packets",
```

```
    "Total Length of Bwd Packets"
```

```
]
```

```
instability_features = [
```

```
    "Flow Bytes/s",
```

```
    "Flow Packets/s",
```

```
    "Fwd Packet Length Std",
```

```
    "Bwd Packet Length Std"
```

```
]
```

```
# =====
```

4) Z-SCORE NORMALIZATION

```
# =====
```

```
for col in coherence_features + instability_features:
```

```
    df[col] = (df[col] - df[col].mean()) / df[col].std()
```

```
# =====
```

5) COMPLEX COHERENCE CONSTRUCTION

```
# =====
```

```
df["x"] = df[coherence_features].mean(axis=1)
```

```
df["y"] = df[instability_features].mean(axis=1)
```

```
df["Q"] = df["x"] + 1j * df["y"]
```

```
df["magnitude"] = np.abs(df["Q"])
```

```
df["phase"] = np.angle(df["Q"])
```

```
# =====
```

6) BALANCED SAMPLING

```
# =====
```

```
min_samples = min(
```

```
    len(df[df["Label"] == "BENIGN"]),
```

```
    len(df[df["Label"] == attack_label]),
```

```
    n_samples
```

```
)
```

```
df_balanced = pd.concat([
```

```
    df[df["Label"] == "BENIGN"].sample(n=min_samples, random_state=seed),
```

```
    df[df["Label"] == attack_label].sample(n=min_samples, random_state=seed)
```

```
])
```

```
print(f"Balanced: {min_samples} BENIGN + {min_samples} {attack_label}")
```

```

# =====

# FIGURE 1: COHERENCE PROJECTION DISTRIBUTION

# =====

theta_0 = np.pi / 4

df_balanced["Score_proj"] = df_balanced["magnitude"] * np.cos(df_balanced["phase"] - theta_0)


fig, ax = plt.subplots(figsize=(6, 4), dpi=300)

colors = {"BENIGN": "#1f77b4", "attack_label": "#d62728"}

for label in ["BENIGN", "attack_label"]:

    subset = df_balanced[df_balanced["Label"] == label]

    ax.hist(subset["Score_proj"], bins=50, alpha=0.6, density=True,

            histtype='stepfilled', label=label, color=colors[label], edgecolor='black', linewidth=0.5)


ax.legend(frameon=False, fontsize=11)

ax.set_xlabel(r"Projected Coherence Score ( $\theta = \pi/4$ )", fontsize=12)

ax.set_ylabel("Probability Density", fontsize=12)

ax.grid(alpha=0.3, linestyle='--')

ax.spines['top'].set_visible(False)

ax.spines['right'].set_visible(False)

plt.tight_layout()

plt.savefig("Fig1_proj_v31.pdf", bbox_inches='tight')

plt.close()

print("Saved: Fig1_proj_v31.pdf")

# =====

```

FIGURE 2: MAGNITUDE DISTRIBUTION

```
# =====  
  
fig, ax = plt.subplots(figsize=(6, 4), dpi=300)  
  
for label in ["BENIGN", attack_label]:  
    subset = df_balanced[df_balanced["Label"] == label]  
    ax.hist(subset["magnitude"], bins=50, alpha=0.6, density=True,  
            histtype='stepfilled', label=label, color=colors[label], edgecolor='black', linewidth=0.5)  
  
ax.legend(frameon=False, fontsize=11)  
ax.set_xlabel("Coherence Magnitude |Q|", fontsize=12)  
ax.set_ylabel("Probability Density", fontsize=12)  
ax.grid(alpha=0.3, linestyle='--')  
ax.spines['top'].set_visible(False)  
ax.spines['right'].set_visible(False)  
plt.tight_layout()  
plt.savefig("Fig2_mag_v31.pdf", bbox_inches='tight')  
plt.close()  
print("Saved: Fig2_mag_v31.pdf")
```

```
# =====
```

FIGURE 3: ROC ANALYSIS

```
# =====
```

```
y_true = (df_balanced["Label"] == attack_label).astype(int)  
y_score = -df_balanced["magnitude"] # Attacks have higher magnitude  
fpr, tpr, thresholds = roc_curve(y_true, y_score)  
roc_auc = auc(fpr, tpr)
```

```
J = tpr - fpr
```

```
best_idx = np.argmax(J)
```

```
fig, ax = plt.subplots(figsize=(5, 5), dpi=300)
```

```
ax.plot(fpr, tpr, lw=2.5, color='#1f77b4', label=f'Vupr-QLI (AUC = {roc_auc:.3f})')
```

```
ax.plot([0, 1], [0, 1], 'k--', lw=1.5, label='Random Classifier')
```

```
ax.scatter(fpr[best_idx], tpr[best_idx], c='red', s=80, zorder=5,  
           edgecolors='black', linewidth=1.5, label="Best Threshold (Youden's J)")
```

```
ax.legend(frameon=False, fontsize=10, loc='lower right')
```

```
ax.set_xlabel("False Positive Rate (FPR)", fontsize=12)
```

```
ax.set_ylabel("True Positive Rate (TPR)", fontsize=12)
```

```
ax.set_xlim([-0.01, 1.01])
```

```
ax.set_ylim([-0.01, 1.01])
```

```
ax.grid(alpha=0.3, linestyle='--')
```

```
ax.set_aspect('equal')
```

```
plt.tight_layout()
```

```
plt.savefig("Fig3_ROC_v31.pdf", bbox_inches='tight')
```

```
plt.close()
```

```
print("Saved: Fig3_ROC_v31.pdf")
```

```
# =====
```

```
# PRINT FINAL METRICS
```

```
# =====
```

```
print("\n" + "="*50)
```

```
print("VUPR-QLI FINAL RESULTS")
```

```
print("="*50)
```

```

print(f"AUC = {roc_auc:.4f}")

print(f"TPR = {tpr[best_idx]:.4f}")

print(f"FPR = {fpr[best_idx]:.4f}")

print(f"Best Threshold = {thresholds[best_idx]:.4f}")

print("="*50)


return roc_auc, tpr[best_idx], fpr[best_idx]


if __name__ == "__main__":

    parser = argparse.ArgumentParser(description="Generate Vupr-QLI Figures 1-3")

    parser.add_argument("--data_dir", type=str, default="data",
                        help="Directory containing CICIDS2017 CSV files")

    parser.add_argument("--attack", type=str, default="DoS Hulk",
                        help="Attack label to detect")

    parser.add_argument("--n_samples", type=int, default=10000,
                        help="Samples per class for balanced dataset")

    parser.add_argument("--seed", type=int, default=0,
                        help="Random seed for reproducibility")

    args = parser.parse_args()

    run_vupr_qli(args.data_dir, args.attack, args.n_samples, args.seed)

```